

ALGORITMICKY TĚŽKÉ PROBLÉMY

Mnoho problémů lze v informatice řešit pomocí polynomiálních algoritmů, tzn. algoritmů běžících v čase $\mathcal{O}(n^k)$ pro nějaké pevné k . Například prvky v poli můžeme řadit různými algoritmy; jednoduchý *BubbleSort* má v nejhorším případě časovou složitost $\mathcal{O}(n^2)$, kde n je počet prvků pole. Také nejkratší cesty v grafu dokážeme hledat polynomiálně, třeba Dijkstrovým algoritmem v čase $\mathcal{O}(n^2 + m)$ při použití pole, kde n je počet vrcholů a m počet hran.¹ S jistou rezervou lze říci, že polynomiální algoritmy bývají prakticky použitelné.

Bohužel ne vždy je situace tak příznivá. Lze narazit na problémy, pro které žádný polynomiální algoritmus neznáme, a u některých dokonce umíme dokázat, že je v polynomiálním čase řešit nelze. Dobrým příkladem úlohy s prokazatelně exponenciálně dlouhým řešením jsou *Hanojské věže*: přesunutí n disků vyžaduje nejméně $2^n - 1$ tahů.

Nás budou zajímat ty nejdůležitější problémy z této oblasti, protože mezi nimi lze nalézt zajímavé vztahy, a posléze si uděláme menší ochutnávku z problematiky P a NP.

1 Problém splnitelnosti

Začneme zdánlivě možná jednoduchým problémem, který však na poli informatiky hraje dosti důležitou roli. Předpokládejme, že máme zadanou nějakou logickou formuli φ o libovolném počtu logických proměnných, označme např. $x_1, x_2, \dots, x_n$. Naší otázkou je, jestli je možné hodnoty proměnných $x_1, x_2, \dots, x_n$ nastavit tak (tj. dosadit hodnoty 0 a 1), aby byla formule splněna², tj. $\varphi(\dots) = 1$. Takovému ohodnocení budeme říkat *splňující*.

SPLNITELNOST OBEČNÉ LOGICKÉ FORMULE

Vstup problému: Logická formule φ .

Výstup problému: 1, pokud je φ splnitelná, jinak 0.

Nejdříve si zopakujeme logické operace a spojky. Mezi ně řadíme $\neg$, $\wedge$, $\vee$, $\implies$, $\iff$.

- **Negace** $\neg x$. *Převrací logickou hodnotu x .*
- **Konjunkce** $a \wedge b$. *Pravdivá, jsou-li obě hodnoty operandů a, b pravdivé.*
- **Disjunkce** $a \vee b$. *Pravdivá, je-li alespoň jedna z hodnot operandů a, b pravdivá.*
- **Implikace** $a \implies b$. *Je-li předpoklad a splněn, je splněn i závěr b . Tj. implikace je nepravdivá, pokud platí a a neplatí b .*
- **Ekvivalence** $a \iff b$. *a platí právě tehdy, když platí b . Tj. ekvivalence je pravdivá, jsou-li hodnoty operandů a, b současně 0 nebo 1.*

Zde konstatujeme, že ve všech příkladech bude mít negace $\neg$ vždy *nejvyšší prioritu* oproti ostatním operacím. Prioritu ostatních operací budeme stanovovat pomocí závorek.

Příklad 1.1. • $\varphi(x) = x \wedge \neg x \dots$ **Není splnitelná**, pro $x = 0$ i $x = 1$ je $\varphi = 0$.

⁽¹⁾U jednoduchého neorientovaného grafu je $m \leq n(n-1)/2$, u jednoduchého orientovaného grafu $m \leq n(n-1)$. V obou případech je tedy m polynomiálně omezeno pomocí n .

⁽²⁾Může se na první pohled zdát, že se jedná o dosti teoretickou záležitost, nicméně později uvidíme, že mnoho jiných problémů má se SAT silnou spojitost.

a	b	$\neg a$	$a \wedge b$	$a \vee b$	$a \implies b$	$a \iff b$
0	0	1	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	1	0	0
1	1	0	1	1	1	1

- $\psi(x) = x \vee \neg x \dots$ **Je splnitelná**, a to jak pro $x = 0$, tak $x = 1$.
- $\chi(a, b, c) = ((a \wedge b) \vee \neg c) \implies (\neg a \wedge \neg c) \dots$ **Je splnitelná**, např. pro $a = 0, b = 0$ a $c = 0$.

V příkladu 1.1 výše se jedná o poměrně jednoduché instance, kdy jsme schopni vidět hned, zda je formule splnitelná. Pokud bychom však uvažili nějakou složitější formuli, problém se již značně zkomplikuje.



Naivní postup pro formuli o n proměnných prozkoumá až 2^n možných ohodnocení. Algoritmus zkoušející všechny možnosti je tak *exponenciální*.

Pro SAT není dodnes známý žádný algoritmus, který by jej uměl řešit pro libovolnou logickou formuli v polynomiálním čase.

1.1 Konjunktivní normální forma

Zkusíme se podívat na formule ve speciálním tvaru, a to tzv. *konjunktivní normální formě*, neboli zkráceně *CNF*.³

Formule v CNF se skládá z **klauzulí**, které obsahují **literály**. Literálem je buď proměnná, nebo její negace.

- Klauzule jsou ve formuli odděleny *logickou spojkou* $\wedge$
- a v každé klauzuli jsou literály odděleny *logickou spojkou* $\vee$.

$$\varphi(\dots) = \underbrace{(\dots \vee \dots \vee \dots)}_{\text{klauzule}} \wedge (\dots \vee \overbrace{x}^{\text{literál}} \vee \dots) \wedge (\dots \vee \overbrace{\neg x}^{\text{literál}} \vee \dots) \wedge (\dots \vee \underbrace{y}_{\text{literál}} \vee \dots) \wedge \dots$$

Příklad 1.2. • $\varphi(x, y, z) = \neg x \wedge (y \vee z) \dots$ **Je v CNF**. Formule φ obsahuje klauzule $\neg x$ (klauzule o jednom literálu) a $y \vee z$.

- $\psi(a, b) = a \wedge b \dots$ **Je v CNF**.
- $\chi(a, b, c, d) = a \wedge (b \vee (c \wedge d)) \dots$ **Není v CNF**, protože výraz $c \wedge d$ není literál.

Poslední formuli lze však převést do CNF: $a \wedge (b \vee c) \wedge (b \vee d)$.

Z příkladu 1.2 se nabízí otázka, zda je možné *libovolnou formuli* převést do CNF. Existuje pro každou formuli ekvivalentní⁴ formule v CNF? Ano.

Věta 1.3. Pro každou logickou formuli ψ existuje ekvivalentní formule ψ' v CNF.

Toto tvrzení lze, pochopitelně, dokázat, ale zde se tím zabývat nebudeme a zkrátka jej přijmeme jako fakt. Podstatná věc, která z toho plyne, je, že pokud je nějaká formule ψ splnitelná, pak je splnitelná i jí ekvivalentní formule ψ'

⁽³⁾Z angl. *conjunctive normal form*.

⁽⁴⁾Formule φ a ψ jsou ekvivalentní, pokud mají při každém ohodnocení společných proměnných stejnou pravdivostní hodnotu.

v CNF a naopak pokud ψ není splnitelná, není splnitelná ani ψ' . Symbolicky

$$\psi \text{ je splnitelná} \iff \psi' \text{ je splnitelná.}$$

1.2 Převod do CNF pomocí tabulky

Podívejme se na způsob, jak lze převést libovolnou logickou formuli do CNF.

Máme obecnou formuli φ s proměnnými $x_1, x_2, \dots, x_n$, pro kterou budeme konstruovat ekvivalentní formuli φ' v CNF. Vycházíme z tabulky pravdivostních hodnot, přičemž pro každé **nepravdivé** ohodnocení vytvoříme *klauzuli* s n literály, kde obecně i -tý literál je

- x_i , pokud v daném ohodnocení je $x_i = 0$,
- jinak $\neg x_i$ (tj. když $x_i = 1$).

Podívejme se na úvod na příklad 1.4 níže.

Příklad 1.4. Máme formuli $\psi(x, y, z) = (x \implies y) \implies z$, pro kterou chceme najít ekvivalentní formuli ψ' v CNF. Nejprve si tedy sestavíme tabulku pravdivostních hodnot. Z tabulky můžeme vidět, že formule ψ je

x	y	z	$x \implies y$	$(x \implies y) \implies z$
0	0	0	1	0
0	0	1	1	1
0	1	0	1	0
0	1	1	1	1
1	0	0	0	1
1	0	1	0	1
1	1	0	1	0
1	1	1	1	1

nepravdivá pro

- $x = 0, y = 0, z = 0$,
- $x = 0, y = 1, z = 0$
- a $x = 1, y = 1, z = 0$.

Tedy celkově sestavíme 3 klauzule:

- $(\overbrace{x}^{x=0} \vee \underbrace{y}_{y=0} \vee \overbrace{z}^{z=0})$,
- $(x \vee \underbrace{\neg y}_{y=1} \vee z)$,
- $(\neg x \vee \neg y \vee z)$.

Výsledná formule v CNF bude mít tvar:

$$\psi'(x, y, z) = (x \vee y \vee z) \wedge (x \vee \neg y \vee z) \wedge (\neg x \vee \neg y \vee z).$$

Čtenář se sám může přesvědčit, že formule ψ a ψ' pro libovolné ohodnocení mají stejnou pravdivostní hodnotu.

Nabízí se otázka: „Proč tento postup funguje?“ Pokud formule v CNF není při určitém ohodnocení pravdivá, alespoň jedna její klauzule není pravdivá. Klauzule obsahující všechny proměnné je nepravdivá právě při jediném ohodnocení: všechny její literály musí být nepravdivé. Pro každé ohodnocení, při němž je původní formule nepravdivá, tedy sestrojíme klauzuli, která zakáže právě toto ohodnocení.

Příklad 1.5. $\chi(x_1, x_2, x_3, x_4) = (x_1 \iff x_2) \vee \neg(x_3 \iff x_4)$.

x_1	x_2	x_3	x_4	$x_1 \iff x_2$	$\neg(x_3 \iff x_4)$	$(x_1 \iff x_2) \vee \neg(x_3 \iff x_4)$
0	0	0	0	1	0	1
0	0	0	1	1	1	1
0	0	1	0	1	1	1
0	0	1	1	1	0	1
0	1	0	0	0	0	0
0	1	0	1	0	1	1
0	1	1	0	0	1	1
0	1	1	1	0	0	0
1	0	0	0	0	0	0
1	0	0	1	0	1	1
1	0	1	0	0	1	1
1	0	1	1	0	0	0
1	1	0	0	1	0	1
1	1	0	1	1	1	1
1	1	1	0	1	1	1
1	1	1	1	1	0	1

$$\chi'(x_1, x_2, x_3, x_4) = (x_1 \vee \neg x_2 \vee x_3 \vee x_4) \wedge (x_1 \vee \neg x_2 \vee \neg x_3 \vee \neg x_4) \wedge (\neg x_1 \vee x_2 \vee x_3 \vee x_4) \wedge (\neg x_1 \vee x_2 \vee \neg x_3 \vee \neg x_4).$$



Problém: Generování tabulky (resp. procházení všech možných ohodnocení) nás přivádí zpět k původnímu problému, a to sice *exponenciální časové složitosti*.

Tento postup nelze obecně zachránit pouhou chytřejší implementací: pokud trváme na ekvivalentní CNF bez nových proměnných, může být výsledná formule exponenciálně delší. Například při roznásobení formule

$$(x_1 \wedge y_1) \vee (x_2 \wedge y_2) \vee \dots \vee (x_n \wedge y_n)$$

vzniká pro každou volbu jednoho literálu z každé dvojice samostatná klauzule, tedy celkem 2^n klauzulí.

1.3 Tseitinova transformace

Značně rychlejší převod do CNF navrhl sovětský matematik *Grigorij S. Cejtin*⁵. Předchozí postup trval na tom, že ve výsledku ponecháme pouze původní proměnné. Tseitinova transformace zavede pomocné proměnné a získá formuli, která sice obecně není s původní formulí ekvivalentní nad rozšířenou množinou proměnných, ale je s ní *stejně splnitelná*.

Postup lze rozdělit do čtyř kroků.

1. Formulí rozložíme na podformule odpovídající uzlům jejího syntaktického stromu.

⁽⁵⁾ Jeho příjmení se v anglických textech obvykle přepisuje jako *Tseitin*; odtud pochází ustálený název transformace.

2. Pro každou složenou podformuli zavedeme novou proměnnou.
3. Pro každou novou proměnnou přidáme klauzule, které vynutí, aby měla stejnou hodnotu jako příslušná podformule.
4. Nakonec přidáme jednotkovou klauzuli tvořenou proměnnou zastupující celou původní formuli; tím požadujeme, aby její hodnota byla 1.

Potřebné klauzule jsou krátké a lze je zapsat přímo. Pro literály nebo pomocné proměnné a, b platí

$$y \iff \neg a \rightsquigarrow (\neg y \vee \neg a) \wedge (y \vee a), \quad (1)$$

$$y \iff (a \wedge b) \rightsquigarrow (\neg y \vee a) \wedge (\neg y \vee b) \wedge (y \vee \neg a \vee \neg b), \quad (2)$$

$$y \iff (a \vee b) \rightsquigarrow (y \vee \neg a) \wedge (y \vee \neg b) \wedge (\neg y \vee a \vee b). \quad (3)$$

Implikaci nejprve přepíšeme pomocí $a \implies b \equiv \neg a \vee b$. Každý výskyt spojky tak přidá nejvýše tři klauzule konstantní délky.

Pojďme se podívat na konkrétní příklad.

Příklad 1.6. Máme formuli $\alpha(p, q, r) = ((p \vee q) \wedge r) \implies \neg p$. Chceme pro ni sestavit Tseitinovu transformaci novou formuli β .

Formule α obsahuje tyto dílčí podformule:

- $\neg p$,
- $p \vee q$,
- $(p \vee q) \wedge r$,
- $a((p \vee q) \wedge r) \implies \neg p$ (původní formule α).

Pro ně zavedeme proměnné y_1, y_2, y_3 a y_4 . Implikaci nejprve přepíšeme jako disjunkci:

- $y_4 \iff \neg p$,
- $y_3 \iff (p \vee q)$,
- $y_2 \iff (y_3 \wedge r)$
- $a \ y_1 \iff (\neg y_2 \vee y_4)$.

Pomocí vztahů (1)–(3) dostaneme formuli v CNF:

$$\begin{aligned} \beta = y_1 & \\ & \wedge (y_1 \vee y_2) \wedge (y_1 \vee \neg y_4) \wedge (\neg y_1 \vee \neg y_2 \vee y_4) \\ & \wedge (\neg y_2 \vee y_3) \wedge (\neg y_2 \vee r) \wedge (y_2 \vee \neg y_3 \vee \neg r) \\ & \wedge (y_3 \vee \neg p) \wedge (y_3 \vee \neg q) \wedge (\neg y_3 \vee p \vee q) \\ & \wedge (\neg y_4 \vee \neg p) \wedge (y_4 \vee p). \end{aligned}$$

Zkusme ohodnocení $p = 1, q = 0$ a $r = 0$. Původní formule je pravdivá:

$$\alpha(1, 0, 0) = ((1 \vee 0) \wedge 0) \implies \neg 1 = 0 \implies 0 = 1.$$

Pomocné proměnné musí nabývat hodnot

- $y_4 = 0$, protože $\neg p = 0$,
- $y_3 = 1$, protože $p \vee q = 1$,

- $y_2 = 0$, protože $y_3 \wedge r = 0$,
- a $y_1 = 1$, protože $\neg y_2 \vee y_4 = 1$.

Po dosazení jsou všechny klauzule formule β splněny.

Je-li původní formule splnitelná, nastavíme každou pomocnou proměnnou podle hodnoty její podformule a splníme všechny nové klauzule. Je-li naopak výsledná formule splnitelná, klauzule vynucují správné hodnoty všech pomocných proměnných a jednotková klauzule pro kořen říká, že původní formule musí být pravdivá. Obě formule jsou tedy stejně splnitelné.



Každá spojka původní formule přidá jednu pomocnou proměnnou a nejvýše tři klauzule. Velikost výsledné formule i doba konstrukce jsou proto lineární ve velikosti původní formule.

2 Problém SAT

V podsekcí 1.1 jsme viděli, že převod obecné formule do CNF „hrubou silou“ je problematický. Nová formule se totiž může až exponenciálně prodloužit. V podsekcí 1.3 jsme si proto ukázali Tseitinovu transformaci, která pomocí nových proměnných sestrojí stejně splnitelnou CNF efektivně. Zkusme si tedy původní problém zjednodušit. Předpokládejme nyní, že formule na vstupu jsou již v CNF. Tento problém je v informatice velmi význačný a označuje se zkratkou SAT (z angl. *satisfiability*, neboli splnitelnost).

SPLNITELNOST LOGICKÉ FORMULE V CNF (SAT)

Vstup problému: Logická formule φ v CNF.

Výstup problému: 1, pokud je φ splnitelná, jinak 0.

Nyní se zaměříme na samotnou splnitelnost vstupní formule. V praxi se používají tzv. *SAT solvery*, které rozhodují, zda je formule v CNF splnitelná. Ukážeme si klasický algoritmus *DPLL*, jehož název připomíná příjmení Martina Davise, Hilaryho Putnama, George Logemanna a Donalda Lovelanda. Jeho základní podoba pochází z počátku 60. let 20. století.

2.1 Algoritmus DPLL

Tento algoritmus je založený na rekurzivním postupu. Na vstupu algoritmu je libovolná formule φ v CNF, kterou algoritmus postupně vyhodnocuje. Využívá dvojici procedur – **Unit propagation** a **Pure literal elimination**.

Unit propagation

Tato procedura hledá v dané formuli φ tzv. *jednotkovou klauzuli*. To je klauzule, která obsahuje **pouze jeden literál**. Například formule ω níže obsahuje jednotkovou klauzuli.

$$\omega(x, y) = (x \vee \neg y) \wedge \underbrace{\neg y}_{\text{jednotková kl.}} \wedge (\neg x \vee y).$$

Pokud se taková klauzule nachází ve formuli, lze toho využít, neboť pro danou proměnnou existuje právě jedno ohodnocení, které tuto klauzuli splní. Obecně pokud formule φ obsahuje jednotkovou klauzuli s proměnnou x , pak

mohou nastat dva případy. Označme φ' zbytek klauzulí ve formuli φ .

1. Pokud má formule tvar $\varphi = x \wedge \varphi'$, pak nutně $x = 1$.
2. Jinak pokud má formule tvar $\varphi = \neg x \wedge \varphi'$, pak musí být $x = 0$.

Fakt, že jsme schopni hned rozhodnout, jaké ohodnocení musí mít daná proměnná, je v tomto případě hodně ku prospěchu, neboť formuli můžeme zjednodušit. Protože ohodnocení dané jednotkové klauzule již známe, můžeme ji z formule φ odstranit. Zároveň můžeme **odstranit všechny klauzule**, které se díky proměnné x vyhodnotí na 1, neboť již nemohou nijak ovlivnit logickou hodnotu zbylých klauzulí.

Např. pokud formule obsahuje literál x v jednotkové klauzuli, pak hodnota proměnné musí být 1. Tzn. klauzule obsahující x se vyhodnotí též na 1. tj.

$$(\dots \vee x \vee \dots) = 1.$$

Naopak pokud obsahuje literál $\neg x$, pak nutně $x = 0$ a tedy

$$(\dots \vee \neg x \vee \dots) = 1.$$

Dále ještě odstraníme výskyty literálu x , resp. $\neg x$ v *opačné polaritě*, tedy negaci původního literálu. Tzn. pokud literál v jednotkové klauzuli byl x , pak opačná polarita literálu je $\neg x$ a naopak. Důvod, proč jej můžeme odstranit, je ten, že v rámci kterékoliv jiné klauzule se daný literál v opačné polaritě vyhodnotí vždy na 0. Protože je však spojen s ostatními literály pomocí $\vee$, nemůže nijak ovlivnit výslednou logickou hodnotu klauzule a tudíž jej můžeme odstranit. Tedy např. pokud $x = 1$, pak můžeme klauzule tvaru

$$(\dots \vee y \vee \underbrace{\neg x}_{\text{odstraníme}} \vee z \vee \dots)$$

zjednodušit na

$$(\dots \vee y \vee z \vee \dots).$$

Podobně pro $x = 0$:

$$(\dots \vee y \vee x \vee z \vee \dots) = (\dots \vee y \vee z \vee \dots).$$

Příklad 2.1. Máme formuli

$$\chi(x, y, z) = (x \vee y) \wedge y \wedge (z \vee \neg y) \wedge (\neg x \vee \neg y \vee \neg z).$$

Aplikujme na ni proceduru *Unit propagation*. Můžeme si všimnout, že klauzule y je jednotková a proměnná y tedy musí být nastavena na 1.

Protože $y = 1$, je zároveň splněna i klauzule $x \vee y$, takže ji můžeme odstranit. Zároveň odstraníme literál $\neg y$ ze zbývajících klauzulí $z \vee \neg y$ a $\neg x \vee \neg y \vee \neg z$. Celkově dostaneme novou zjednodušenou formuli

$$\chi'(x, y, z) = z \wedge (\neg x \vee \neg z).$$

Pure literal elimination

Ve vstupní formuli φ může nastat situace, kdy se zde určitá proměnná nachází pouze v jedné polaritě. Tzn. pokud se ve formuli nachází proměnná x , pak se v ní nikde nenachází $\neg x$ a naopak pokud se v ní nachází $\neg x$, pak v ní nikde není x . V obou případech můžeme rovnou určit hodnotu dané proměnné, neboť tím splníme všechny klauzule, v nichž se vyskytuje. Ty můžeme odstranit, neboť jsou tím pádem splněny.

$$\dots \wedge \underbrace{(\dots \vee x \vee \dots)}_1 \wedge \dots \wedge \underbrace{(\dots \vee x \vee \dots)}_1 \wedge \dots$$

Podobně pro negaci, tj.

$$\dots \wedge \overbrace{(\dots \vee \neg \underbrace{x}_0 \vee \dots)}^1 \wedge \dots \wedge \overbrace{(\dots \vee \neg \underbrace{x}_0 \vee \dots)}^1 \wedge \dots$$

Poté, co jsme si vysvětlili základní dvojici procedur, se nyní nabízí trojice případů, které je ještě třeba prozkoumat.

- *Co když postupným odstraňováním proměnných mi ve formuli vznikne prázdná klauzule?* Taková situace může nastat v případě procedury Unit propagation, kdy odstraňujeme proměnné v opačné polaritě. Pokud nastane situace, že je nějaká klauzule prázdná, pak to znamená, že daná klauzule pod zvoleným ohodnocením **není splnitelná** a tudíž ani celá formule. V takovou chvíli můžeme prohlásit formuli za **nesplnitelnou pod daným ohodnocením**⁶.
- *Co když odstraníme postupně všechny klauzule?* K takové situaci může dojít, pokud jsou všechny klauzule splněny. V takovou chvíli určitě víme, že je původní formule **splnitelná**.
- *Co když po aplikaci obou procedur stále nevíme, zda je φ splnitelná?* V takovou chvíli nám nezbyvá nic jiného než si zvolit nějakou dosud neohodnocenou proměnnou a prozkoumat obě možnosti jejího ohodnocení. Vybereme tedy proměnnou x a zkusíme ji nastavit jednou na hodnotu 1 a podruhé na hodnotu 0. Tím můžeme formuli φ zjednodušit a na nově vzniklou formuli φ' rekurzivně aplikovat stejný algoritmus. Pokud lze splnit zbytek formule φ' , pak lze splnit i původní formuli φ .

Toho můžeme dosáhnout tak, že k formuli φ přidáme uměle jednotkovou klauzuli x , resp. v druhém případě $\neg x$. Tuto jednotkovou klauzuli si pak vybere procedura Unit propagation, která proměnnou ohodnotí. Zavoláme tedy algoritmus DPLL rekurzivně na formule

$$\varphi \wedge x \quad \text{a} \quad \varphi \wedge \neg x.$$

S těmito informacemi můžeme dát dohromady pseudokód algoritmu.

Algoritmus 2.1: DPLL

Vstup: Formule φ v CNF

```

1 while existuje jednotková klauzule ( $\ell$ ) ve formuli  $\varphi$  do
2    $\varphi \leftarrow \text{UnitPropagation}(\ell, \varphi)$ 
3 while existuje literál  $\ell$  v jediné polaritě ve  $\varphi$  do
4    $\varphi \leftarrow \text{PureLiteralElimination}(\ell, \varphi)$ 
   // Všechny klauzule jsou pravdivé
5 if je  $\varphi$  prázdná then return 1;
   // Všechny literály v klauzuli jsou nepravdivé
6 if obsahuje  $\varphi$  prázdnou klauzuli then return 0;
   // Zkusíme obě ohodnocení proměnné  $x$ 
7 Vyber dosud neohodnocenou proměnnou  $x$ 
8 return DPLL( $\varphi \wedge x$ )  $\vee$  DPLL( $\varphi \wedge \neg x$ )
```

Výstup: 1, je-li formule φ splnitelná, jinak 0.

Zkusme si algoritmus odsimulovat na příkladu.

Příklad 2.2. Máme formuli

$$\gamma(x_1, x_2, x_3, x_4) = (x_1 \vee x_3) \wedge \neg x_4 \wedge (\neg x_2 \vee \neg x_3) \wedge x_1 \wedge (\neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_3 \vee x_3).$$

⁽⁶⁾ Je potřeba si uvědomit, že ohodnocení proměnných si během výpočtu volíme, tedy v takovém případě to ještě nutně neznamená, že je formule celkově nesplnitelná. Pouze víme, že dané ohodnocení určitě není splňující

Chceme pomocí algoritmu DPLL rozhodnout, zda je splnitelná.

Když se podíváme na pseudokód 2.1 výše, tak první máme aplikovat proceduru *Unit propagation*. Hledáme tedy jednotkové klauzule ve formuli γ , dokud nějaké existují.

- První jednotková klauzule, které si můžeme všimnout, je $\neg x_4$. Hodnota proměnné x_4 tedy nutně musí být 0. Klauzuli tedy odstraníme. Proměnná se nachází ve formuli pouze jednou, tedy daný literál v opačné polaritě nikde odstraňovat nemusíme. Máme nyní formuli

$$\gamma'(x_1, x_2, x_3, x_4) = (x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3) \wedge x_1 \wedge (\neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_3 \vee x_3).$$

- Další jednotková klauzule je x_1 . Proměnná x_1 musí být nutně 1 a klauzuli můžeme odstranit. Dále si však můžeme všimnout, že tím pádem splněna i klauzule $(x_1 \vee x_3)$. Tu můžeme též odstranit. Literál x_1 se dále nachází v opačné polaritě v klauzuli $(\neg x_1 \vee \neg x_3 \vee x_3)$. Literál $\neg x_1$ můžeme také odstranit z formule. Tím se nám formule γ' zjednoduší na

$$\gamma''(x_1, x_2, x_3, x_4) = (\neg x_2 \vee \neg x_3) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_3 \vee x_3).$$

Nově vzniklá formule γ'' již žádné další jednotkové klauzule neobsahuje. Dále tedy můžeme aplikovat proceduru *Pure literal elimination*.

- Formule γ'' obsahuje pouze jeden literál v jediné polaritě, a to $\neg x_2$. Tím pádem hodnotu x_2 můžeme nastavit na 0, čímž splníme dvojici klauzulí $(\neg x_2 \vee \neg x_3)$ a $(\neg x_2 \vee x_3)$. Ty můžeme odstranit, čímž dostaneme formuli

$$\gamma'''(x_1, x_2, x_3, x_4) = \neg x_3 \vee x_3.$$

Protože formule γ''' není ani prázdná, ani neobsahuje prázdnou klauzuli, vybereme náhodně jeden literál a prozkoumáme jeho obě možná ohodnocení. V tomto případě nám zbývají pouze literály x_3 a $\neg x_3$. Vyberme např. literál $\neg x_3$. Nyní máme sestavit dvojici nových formulí $(\neg x_3 \vee x_3) \wedge \neg x_3$ a $(\neg x_3 \vee x_3) \wedge x_3$, na které rekurzivně zavoláme DPLL.

- **Formule** $(\neg x_3 \vee x_3) \wedge \neg x_3$. Opět začneme aplikací Unit propagation. K původní formuli γ''' jsme uměle přidali jednotkovou klauzuli $\neg x_3$. Z ní je zjevné, že x_3 musí být 0. Tím zároveň splníme i klauzuli $(\neg x_3 \vee x_3)$, kterou můžeme odstranit, čímž dostáváme prázdnou formuli. Pure literal elimination aplikovat nemůžeme. Protože je nově vzniklá formule prázdná, vrátíme na výstup 1.
- **Formule** $(\neg x_3 \vee x_3) \wedge x_3$. I zde začneme eliminací jednotkových klauzulí. Tu zde představuje literál x_3 , jehož ohodnocení tedy musí být 1. Tím opět splníme i klauzuli $(\neg x_3 \vee x_3)$, jejímž odstraněním dostaneme zase prázdnou formuli. Tedy opět vrátíme 1.

Obě ohodnocení pro x_3 jsou tedy splňující a tedy i původní formule γ je *splnitelná*.

Ukažme si ještě jeden jednodušší příklad.

Příklad 2.3. Mějme formuli $\tau(x, y, z) = x \wedge (\neg x \vee y \vee z) \wedge \neg x$. Chceme opět pomocí DPLL určit splnitelnost.

Jako první jednotkovou klauzuli můžeme vzít x . Tedy můžeme ji odstranit společně se všemi výskyty daného literálu v opačné polaritě, tj. $\neg x$. Tzn. dostaneme formuli

$$\tau'(x, y, z) = (y \vee z) \wedge ().$$

Všimněte si, že odstraněním literálu $\neg x$ jsme dostali ve formuli prázdnou klauzuli⁷ (). Aplikací Pure literal elimination se zbavíme i literálů y a z .

$$\tau''(x, y, z) = ()$$

Formule τ'' obsahuje pouze prázdnou klauzuli. V tuto chvíli algoritmus prohlásí formuli τ za *nesplnitelnou*.

⁽⁷⁾ Z jednotkových klauzulí jsme odebrali její jediný literál v rámci procedury Unit propagation; klauzule samotná ve formuli zůstala.

Poslední otázka, která se nabízí, je (jako obvykle) časová složitost algoritmu. Vezmeme-li v potaz nejhorší případ, časová složitost bohužel nevychází úplně příznivě.

Věta 2.4 (Složitost DPLL). Nechť formule φ obsahuje v proměnných a její délka je s . Jednoduchá implementace DPLL doběhne v čase $\mathcal{O}(s2^v)$. Při kopírování formule v každé úrovni rekurze spotřebuje nejvýše $\mathcal{O}(sv)$ pomocné paměti.

Důkaz. V nejhorším případě nepomůže ani jednotková propagace, ani eliminace čistých literálů. Algoritmus pak zvolí proměnnou a vytvoří dvě větve, jednu pro hodnotu 0 a druhou pro hodnotu 1. Hloubka stromu je nejvýše v a počet jeho uzlů

$$1 + 2 + 4 + \dots + 2^v = 2^{v+1} - 1 = \mathcal{O}(2^v).$$

V každém uzlu lze naivně projít celou formuli v čase $\mathcal{O}(s)$, odkud plyne čas $\mathcal{O}(s2^v)$. Rekurse probíhá do hloubky, takže současně uchováваме nejvýše v kopií formule délky nejvýše s ; to dává $\mathcal{O}(sv)$ pomocné paměti. $\square$



Praktické SAT solvery formulí při každém volání celou nekopírují. Změny zapisují do pomocných struktur a při návratu je vracejí zpět, takže paměťový odhad lze podstatně zlepšit.

Se exponenciální časovou složitostí se sotva spokojíme, avšak v tomto případě je dobré si říci, že zkoumáme složitost v nejhorším případě a v praxi bývají zkoumané logické formule poměrně rychle řešitelné pomocí DPLL.



Problém je však dodnes ten, že na rozhodnutí splnitelnosti logické formule (i těch v CNF) není známý žádný algoritmus, který by běžel v polynomiálním čase, tj. $\mathcal{O}(n^k)$, pro nějaké k . K tomuto tématu se ještě vrátíme v sekci 6.

3 Problém 3-SAT

Zkusme nyní původní problém ještě omezit. Přidáme podmínku, že každá klauzule může obsahovat nejvýše tři literály. Tím dostáváme variantu problému SAT známou jako 3-SAT. O formuli, která splňuje uvedenou podmínku, budeme říkat, že je v 3-CNF.

SPLNITELNOST LOGICKÉ FORMULE V 3-CNF (3-SAT)

Vstup problému: Formule φ v 3-CNF

Výstup problému: 1, je-li φ splnitelná, jinak 0.

Nabízí se otázka, zda jsme si skutečně pomohli. Zdánlivě ano, protože máme kratší klauzule. Překvapivě však 3-SAT zůstává stejně obtížný v tom smyslu, že SAT lze na 3-SAT převést v polynomiálním čase. Pro libovolnou instanci SAT sestrojíme instanci 3-SAT, která je splnitelná právě tehdy, když byla splnitelná původní formule.

3.1 Převod SAT na 3-SAT

Popíšeme si nyní obecný způsob, jak převést libovolnou formuli v CNF (instance problému SAT) na formuli v 3-CNF (instance problému 3-SAT). Předpokládejme, že máme zadanou nějakou formuli φ v CNF a chceme pro ni sestřit formuli ψ v 3-CNF, takovou, aby byla splnitelná právě tehdy, když je splnitelná i původní formule φ . Pro formuli φ mohou nastat 2 případy:

1. formule obsahuje klauzule délky 3 a kratší,
2. nebo ve formuli existuje klauzule délky $k > 3$.

V prvním případě není co řešit, neboť φ již je v 3-CNF, tedy stačí položit $\psi = \varphi$ a jsme hotovi. V druhém případě musíme dlouhou klauzuli o k literálech rozložit na kratší. Tuto klauzuli⁸ rozložíme na dvojici nových klauzulí pomocí nové proměnné x . Označme si literály dané dlouhé klauzule $\ell_1, \ell_2, \dots, \ell_k$. První dvojici ℓ_1 a ℓ_2 si označíme α a zbytek β .

$$\underbrace{\ell_1 \vee \ell_2}_{\alpha} \vee \overbrace{\ell_3 \vee \dots \vee \ell_k}^{\beta} = \alpha \vee \beta$$

Nyní přidáme novou proměnnou x a klauzuli $\alpha \vee \beta$ rozdělíme na dvojici klauzulí následovně:

$$(\alpha \vee x) \wedge (\beta \vee \neg x).$$

Podívejme se nyní na délky nových klauzulí $\alpha \vee x$ a $\beta \vee \neg x$.

- Klauzule $\alpha \vee x = \ell_1 \vee \ell_2 \vee x$ obsahuje 3 literály, takže s ní dále nic provádět nemusíme.
- Klauzule $\beta \vee \neg x = \ell_3 \vee \dots \vee \ell_k \vee \neg x$ obsahuje $k - 1$ literálů. To znamená, že jsme zkrátali původní klauzuli o jeden literál. Pokud $k - 1 > 3$, můžeme postup pro tuto klauzuli opakovat.

Nyní zbývá dořešit, jak to bude s hodnotou proměnné x . Podobně jako u Tseitinovy transformace (viz podsekcce 1.3), i zde bude její hodnota záviset na hodnotách ostatních proměnných, konkrétně výrazu α .

- Pokud $\alpha = 1$, pak nastavíme $x = 0$. Původní klauzule $\ell_1 \vee \dots \vee \ell_k = \alpha \vee \beta$ je splněna díky α . Nová dvojice klauzulí je pak splněna pomocí α a x :

$$(\alpha \vee x) \wedge (\beta \vee \neg x) = (1 \vee 0) \wedge (\beta \vee 1) = 1 \wedge 1 = 1.$$

- Pokud $\alpha = 0$, pak nastavíme $x = 1$. Klauzule $\alpha \vee \beta$ musí být splněna díky β . Potom platí:

$$(\alpha \vee x) \wedge (\beta \vee \neg x) = (0 \vee 1) \wedge (1 \vee 0) = 1 \wedge 1 = 1.$$

Pokud by však $\beta = 0$, pak by původní klauzule $\alpha \vee \beta$ nebyla splněna a potom

$$(\alpha \vee x) \wedge (\beta \vee \neg x) = (0 \vee 1) \wedge (0 \vee 0) = 1 \wedge 0 = 0,$$

tzn. ani nová dvojice klauzulí by nebyla splněna, což chceme.

Platí i opačný směr: kdyby byla $\alpha \vee \beta$ nepravdivá, muselo by být $\alpha = \beta = 0$. Nové klauzule by pak vyžadovaly současně $x = 1$ a $\neg x = 1$, což není možné. Nová dvojice je tedy splnitelná právě tehdy, když je splnitelná původní klauzule. Tím jsme dostali obecný postup pro převod formule z CNF do 3-CNF.

Příklad 3.1. Máme formuli

$$\varphi(a, b, c, d, e, f) = a \vee b \vee \neg c \vee d \vee \neg e \vee f$$

o jedné klauzuli, kterou chceme převést do 3-CNF. K tomu si pořídíme novou proměnnou x . Podle postupu popsaného výše sestavíme dvojici nových klauzulí $\alpha \vee x$ a $\beta \vee \neg x$.

$$\underbrace{a \vee b}_{\alpha} \vee \overbrace{\neg c \vee d \vee \neg e \vee f}^{\beta}$$

⁽⁸⁾ Pokud φ obsahuje více dlouhých klauzulí, postup opakujeme pro každou z nich zvlášť.

Tím dostáváme novou formuli

$$\varphi' = (a \vee b \vee x) \wedge (\neg c \vee d \vee \neg e \vee f \vee \neg x).$$

První klauzule $a \vee b \vee x$ obsahuje tři literály, avšak druhá klauzule je stále dlouhá (všimněte si ale, že obsahuje o jeden literál méně než původní klauzule). Postup tedy opakujeme.

Vytvoříme si opět novou proměnnou, označme ji y . Podobně jako výše i zde rozložíme danou klauzuli na dvě kratší.

$$(\underbrace{\neg c \vee d}_{\alpha'} \vee y) \wedge (\underbrace{\neg e \vee f \vee \neg x \vee \neg y}_{\beta'}).$$

Tedy dostáváme formuli

$$\varphi'' = (a \vee b \vee x) \wedge (\neg c \vee d \vee y) \wedge (\neg e \vee f \vee \neg x \vee \neg y).$$

Třetí klauzule je stále ještě dlouhá, tedy postup zopakujeme ještě jednou s novou proměnnou z .

$$(\underbrace{\neg e \vee f \vee z}_{\alpha''}) \wedge (\underbrace{\neg x \vee \neg y \vee \neg z}_{\beta''}).$$

Celkově dostáváme formuli, která je již v 3-CNF:

$$\psi = (a \vee b \vee x) \wedge (\neg c \vee d \vee y) \wedge (\neg e \vee f \vee z) \wedge (\neg x \vee \neg y \vee \neg z).$$

Nyní si zkusíme určit hodnoty pomocných proměnných x, y, z na základě zadaného ohodnocení původních proměnných. Zkusme

$$a = 0, \quad b = 0, \quad c = 0, \quad d = 0, \quad e = 1, \quad f = 0.$$

Původní formule je díky literálu $\neg c$ splněna.

- Protože $a \vee b = 0$, položíme $x = 1$.
- Dále máme $\neg c \vee d = 1$, tedy položíme $y = 0$.
- Nakonec $\neg e \vee f = 0$, a proto položíme $z = 1$.

Můžeme se přesvědčit, že nová formule ψ je v tomto ohodnocení také splněna:

$$\begin{aligned} \psi &= (0 \vee 0 \vee 1) \wedge (1 \vee 0 \vee 0) \wedge (0 \vee 0 \vee 1) \wedge (0 \vee 1 \vee 0) \\ &= 1 \wedge 1 \wedge 1 \wedge 1 \\ &= 1. \end{aligned}$$

Tímto způsobem jsme ukázali, že problém SAT lze “převést” na 3-SAT. Opačný směr je triviální, protože každá formule v 3-CNF už je i v CNF, takže nic převádět nemusíme.

Zkusme se zamyslet nad rychlostí převodu. Klauzuli o $k > 3$ literálech nahradíme $k - 2$ klauzulemi a použijeme $k - 3$ nových proměnných. Celkový počet vytvořených symbolů je tedy lineární v délce původní formule.

Z toho plyne, že kdybychom uměli řešit 3-SAT v polynomiálním čase, uměli bychom polynomiálně řešit i SAT: nejprve bychom lineárně sestrojili formuli v 3-CNF a poté na ni spustili předpokládaný polynomiální algoritmus pro 3-SAT. Samotný převod je lineární; *to však neznamená, že lineárně umíme rozhodnout splnitelnost výsledné formule.*

Toto byla ukázka tzv. **polynomiálního převodu** mezi problémy. Pojďme si toto trochu rigorózněji definovat.

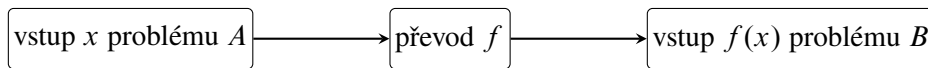
Definice 3.2 (Polynomiální převoditelnost). Řekneme, že libovolný problém A je polynomiálně převoditelný na problém B , píšeme $A \rightarrow B$, pokud každý vstup x problému A lze převést v polynomiálním čase na vstup x' problému B , přičemž $A(x) = B(x')$.

Co tato definice říká? Pokud platí $A \rightarrow B$, pak odpověď na vstup x' je zároveň i odpovědí na původní vstup x . V našem případě je formule v CNF φ splnitelná (to je naše x), právě když je nová formule v 3-CNF ψ (to je x') splnitelná.

Zároveň $A \rightarrow B$ znamená, že problém B je alespoň tak těžký jako problém A .⁹



Ze směru převodu plyne následující podmíněné tvrzení: pokud $A \rightarrow B$ a problém B lze řešit polynomiálně, pak lze polynomiálně řešit i A . Obrácený závěr obecně neplatí. Pokud by navíc bylo dokázáno, že A polynomiálně řešit nelze, pak by z $A \rightarrow B$ plynulo, že polynomiálně nelze řešit ani B . U SAT však takový dolní odhad zatím dokázán není.



$$x \in A \iff f(x) \in B$$

Obrázek 1: Schéma polynomiálního převodu.

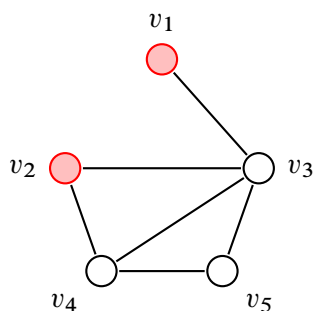
4 Problém nezávislé množiny v grafu

Od problému SAT, resp. 3-SAT se na chvíli přesuneme ke zdánlivě odlišnému problému z teorie grafů. Později si ukážeme i jeho praktické použití v podsekcí 4.2. Nejdříve však bude dobré začít definicí.

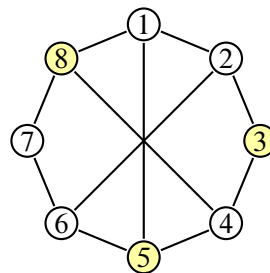
Definice 4.1 (Nezávislá množina). Mějme libovolný neorientovaný graf $G = (V, E)$. Množina $N \subseteq V$ je *nezávislá*, pokud žádné dva různé vrcholy z N nejsou spojeny hranou.

Definice 4.1 jednoduše říká, že jakákoliv vybraná množina vrcholů z daného grafu je nezávislá, pokud mezi všemi vybranými vrcholy nevede hrana. Podívejme se na konkrétní příklad.

Příklad 4.2. Nezávislé množiny M_1 a M_2 v grafech G_1 a G_2 (viz obrázky 2a a 2b).



(a) Nezávislá množina vrcholů $M_1 = \{v_1, v_2\}$ v G_1



(b) Nezávislá množina vrcholů $M_2 = \{3, 5, 8\}$ v G_2

⁽⁹⁾ Mnemotechnická pomůcka: „Obtížnost teče ve směru šipky.“

Zde poznamenejme, že na pouhou existenci neprázdné nezávislé množiny nemá smysl se ptát, neboť v libovolném neprázdném grafu existuje nezávislá množina: libovolná jednovrcholová množina je z definice nezávislá. Zajímavější otázkou je spíše, zda v zadaném grafu existuje nezávislá množina, která má alespoň určitou velikost (tzn. počet vrcholů).

NEZÁVISLÁ MNOŽINA V GRAFU (NzMNA)

Vstup problému: Graf $G = (V, E)$ a číslo $k \in \mathbb{N}$.

Výstup problému: 1, pokud v grafu G existuje nezávislá množina velikosti alespoň k , jinak 0.

Jak později uvidíme, otázka existence nezávislé má i své aplikace při řešení praktických problémů (opět viz podsekcce 4.2), avšak ještě než tak učiníme, zkusme se podívat na souvislosti s problémy, které již známe.

4.1 Převod 3-SAT na NzMna

Mezi dvojicí problémů NzMna a 3-SAT existuje přímá souvislost oběma směry. Problém 3-SAT lze tedy převést na problém nezávislé množiny a naopak. Pro účely tohoto textu si však ukážeme pouze převod jedním směrem, ten druhý je již podstatně složitější¹⁰.

Začínáme s formulí φ , která je ve formě 3-CNF. Uvažujme, že obsahuje obecně k klauzulí, kde každá obsahuje maximálně 3 literály. Chceme pro tuto formuli sestavit graf G a číslo n takové, že v něm existuje nezávislá množina velikosti alespoň n .

$$\varphi = \underbrace{(\dots)}_{\text{max. 3 literály}} \wedge (\dots) \wedge \dots \wedge (\dots)$$

Aby byla formule φ splněná, musí být splněny všechny její klauzule. K tomu je potřeba, aby v každé klauzuli existoval alespoň jeden literál, který ji bude splňovat (může jich být i více). Z každé klauzule tedy vybereme jeden literál, který ji bude splňovat. Je však nutné upozornit, že nesmíme vybírat konfliktně, tedy např. nelze v jedné klauzuli vybrat jako splňující literál x a v jiné klauzuli $\neg x$.

$$(\dots) \wedge (\dots \vee \underbrace{x}_{\text{vybereme}} \vee \dots) \wedge \dots \wedge (\dots \vee \overbrace{\neg x}^{\text{nesmíme vybrat}} \vee \dots) \wedge \dots$$

Pro každý výskyt literálu vytvoříme v novém grafu jeden vrchol. Vrcholy náležející stejné klauzuli navzájem všechny spojíme; pro tříliterálovou klauzuli tak vznikne trojúhelník, pro kratší klauzuli hrana nebo jediný vrchol. Nakonec spojíme hranou každý výskyt literálu x s každým výskytem opačného literálu $\neg x$ v jiné klauzuli. Například pro formuli

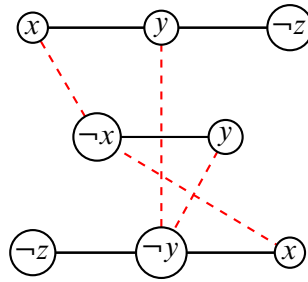
$$\psi(x, y, z) = (x \vee y \vee \neg z) \wedge (\neg x \vee y) \wedge (\neg z \vee \neg y \vee x)$$

je příslušný graf znázorněný na obrázku 3.

Pojďme nyní ověřit obě strany konstrukce. Je-li formule splnitelná, vybereme z každé klauzule jeden pravdivý literál. Získané vrcholy pocházejí z různých klauzulí a nemohou obsahovat současně x i $\neg x$, takže mezi nimi nevede žádná hrana. Tvoří proto nezávislou množinu velikosti k .

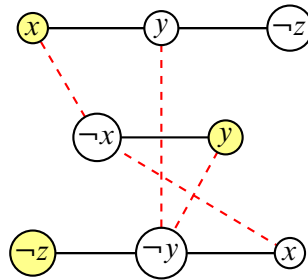
Naopak nechť graf obsahuje nezávislou množinu velikosti k . Z každé z k klauzulí může obsahovat nejvýše jeden vrchol, protože vrcholy jedné klauzule jsou navzájem spojené. Musí tedy obsahovat právě jeden vrchol z každé klauzule. Vybrané literály si navzájem neodporují, jinak by jejich vrcholy byly spojeny hranou. Nastavíme proměnné tak, aby byly všechny vybrané literály pravdivé; tím splníme každou klauzuli.

⁽¹⁰⁾ Pro zájemce viz kniha *Průvodce labyrintem algoritmů* (2. vydání), str. 471



Obrázek 3: Ukázka sestrojeného grafu pro formuli ψ .

Pro již zmíněnou formuli ψ , která je splnitelná např. pomocí ohodnocení $x = 1, y = 1$ a $z = 0$, je příslušná nezávislá množina znázorněna na obrázku 4. Vybrané splňující literály jednotlivých klauzulí jsou x, y a $\neg z$.



Obrázek 4: Ukázka nezávislé množiny příslušné splňujícím literálům pro formuli ψ .

Jako poslední musíme určit cílové číslo n . Protože formule obsahuje k klauzulí a z každé vybíráme právě jeden literál, položíme $n = k$.

K převodu NzMna na 3-SAT přidáme jen krátký komentář. Idea převodu je využití klasického SATu. Zadaný graf G a číslo k se převede na formuli v CNF a ta se následně převede do 3-CNF. Problém SAT tak využijeme v jistém smyslu jako “prostředníka” (viz obrázek 5).



Obrázek 5: Idea převodu NzMna $\rightarrow$ 3-SAT.

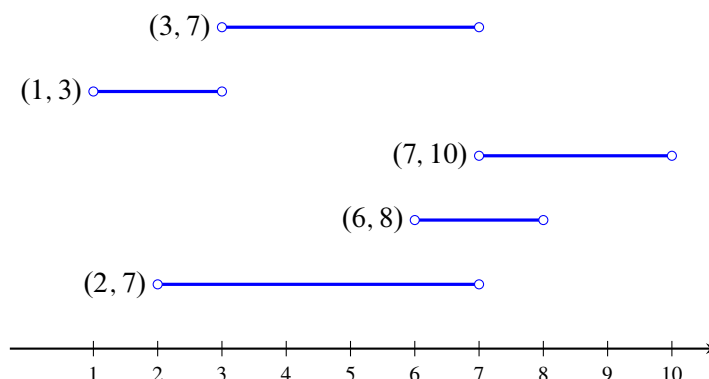
4.2 Aplikace NzMna – barvení intervalového grafu

Nyní se podíváme na jednu z aplikací nezávislé množiny v praktických problémech.

Představme si, že máme n vyučovacích hodin ve škole, které chceme rozvrhnout do co nejmenšího počtu učeben tak, aby se žádné dvě z nich nepřekrývaly. Každá vyučovací hodina má čas začátku a čas svého konce. Matematicky toto můžeme reprezentovat pomocí intervalu (a, b) , kde a je čas začátku a b je čas konce. Jednotlivé hodiny tedy můžeme reprezentovat pomocí intervalů $(a_1, b_1), (a_2, b_2), (a_3, b_3), \dots, (a_n, b_n)$. V tomto případě záměrně volíme otevřené intervaly. Později uvidíme, proč se nám toto hodí.

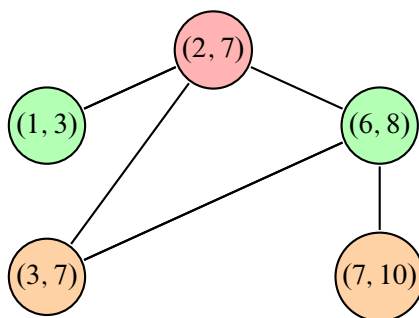
Dané hodiny tedy chceme rozdělit do co nejmenšího počtu učeben. Každou z učeben si můžeme označit nějakou barvou. Nyní sestojíme tzv. **intervalový graf**. Jeho vrcholy představují dané vyučovací hodiny, resp. příslušné intervaly, přičemž mezi dvojicí vrcholů vede hrana, právě když dané intervaly mají neprázdný průnik, tj. obecně $(a_i, b_i) \cap (a_j, b_j) \neq \emptyset$ (to znamená, že se hodiny svým časem kryjí). Zde si všimněme, proč se nám hodí pracovat

s otevřenými intervaly. Pokud by dvojice hodin navazovala těsně po sobě ve stejné učebně, např. $(7, 9)$ a $(9, 10)$, pak bychom takovou situaci také mohli shledat přípustnou. Pokud bychom však používali uzavřené intervaly, pak by průnik vycházel neprázdný. Např. viz obrázek 6. Nyní každý vrchol obarvíme barvou nějaké učebny. Přitom



Obrázek 6: Příklad intervalového grafu pro intervaly $(2, 7)$, $(6, 8)$, $(7, 10)$, $(1, 3)$, $(3, 7)$.

dvojice vyučovacích hodin nesmí být rozvržena do stejné učebny, pokud se časově kryjí. To znamená, že sousední vrcholy musí mít vždy různé barvy (viz obrázek 7). Zde si všimněte, že libovolná skupina vrcholů obarvená stejnou



Obrázek 7: Obarvení intervalového grafu pro intervaly $(2, 7)$, $(6, 8)$, $(7, 10)$, $(1, 3)$, $(3, 7)$.

barvou tvoří nezávislou množinu.

V praxi je efektivní rozvrhování běžnou záležitostí, například při přidělování úloh procesorům.



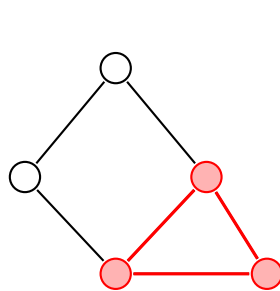
Barvení obecného grafu je algoritmicky těžké, ale intervalové grafy mají zvláštní strukturu. Minimální počet učeben lze určit polynomiálním hladovým algoritmem: hodiny zpracujeme podle času začátku a použijeme nejdříve uvolněnou dostupnou učebnu; není-li žádná volná, otevřeme novou. Tento příklad tedy ukazuje, že omezená varianta obecně těžkého problému může být naopak dobře řešitelná.

5 Problém kliky v grafu

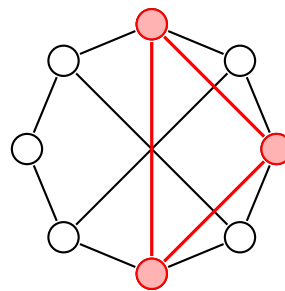
Jako poslední si ukážeme ještě jeden typ problému, který (jak uvidíme) přímo souvisí s problémem nezávislé množiny. Nejdříve si však připomeňme některé důležité termíny z teorie grafů. Jako první si definujeme tzv. *kliku*.

Definice 5.1 (Klika). Nechť je dán neorientovaný graf $G = (V, E)$. Množinu $K \subseteq V$ nazveme *klikou*, pokud jsou každé dva různé vrcholy z K spojeny hranou.

Příklad 5.2. Příklady klik v grafech G_1 a G_2 , viz obrázky 8a a 8b.



(a) Klika v grafu G_1 .



(b) Klika v grafu G_2 .

Podobně jako u nezávislé množiny, i zde platí, že každý neprázdný graf jistě obsahuje kliku (např. samostatný vrchol je z definice klika). Bude nás tedy opět zajímat, zda v zadaném grafu existuje klika určité velikosti.

PROBLÉM KLIKY V GRAFU (KLIKA)

Vstup problému: Graf $G = (V, E)$ a číslo $k \in \mathbb{N}$.

Výstup problému: 1, pokud v G existuje klika velikosti alespoň k , jinak 0.

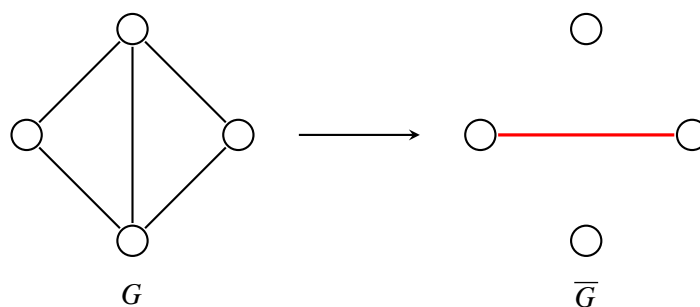
Nyní si ukážeme, že ve skutečnosti problém kliky je ekvivalentní s problémem nezávislé množiny. K tomu si však definujeme ještě jeden termín.

Definice 5.3 (Doplňek grafu). Je-li $G = (V, E)$ jednoduchý neorientovaný graf, pak jeho *doplňkem* je graf $\overline{G} = (V, \overline{E})$, kde pro každé dva různé vrcholy $u, v \in V$ platí

$$\{u, v\} \in \overline{E} \iff \{u, v\} \notin E.$$

Co tato definice vlastně říká? Jednoduše to, že doplněk grafu G obsahuje právě ty hrany, které v původním grafu chybí. Podívejme se na příklady.

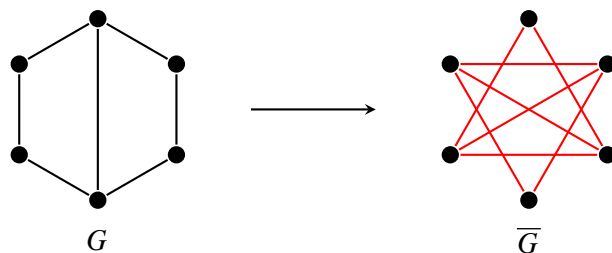
Příklad 5.4. Příklady doplňků grafů, viz obrázky 9 a 10.



Obrázek 9: Doplňek grafu na čtyřech vrcholech.

5.1 Převod Klika na NzMna

Jako poslední se zamyslíme, jaká je tedy souvislost mezi problémem nezávislé množiny a kliky v grafu. Předpokládejme, že máme zadaný graf G a číslo $k \in \mathbb{N}$. Chceme sestavit graf G' a číslo k' takové, že v G existuje klika velikosti alespoň k právě tehdy, když v novém grafu G' existuje nezávislá množina velikosti alespoň k' . To zařídíme velice jednoduše.



Obrázek 10: Doplněk prasátkového grafu.

Pro daný graf G sestojíme jeho doplněk $\overline{G}$. Pokud je K klikou v G , pak každé dva různé vrcholy z K jsou v G spojeny hranou, a proto v $\overline{G}$ spojeny nejsou. Množina K je tedy nezávislá v $\overline{G}$. Stejný argument funguje i obráceně.

Tedy celkově stačí položit $G' = \overline{G}$ a $k' = k$ a jsme hotovi. Opačný převod provedeme naprosto stejně.

6 Třídy problémů P a NP

V této sekci se nyní ohledneme trochu zpět a podíváme se na celou problematiku z více abstraktního hlediska. Naším cílem je představit si dvojici základních tříd problémů a vysvětlit si jejich podstatu v současném světě informatiky. Jako první se trochu blíže podíváme na problémy, které jsme si již představili.

6.1 Rozbor předchozích problémů

V předchozích sekcích jsme si představili některé základní problémy. Všimněte si, že vždy jsme se ptali na to, zda nějaký objekt splňuje určitou vlastnost. Např. u logické formule jsme se ptali, zdali je či není splnitelná. U grafů jsme se ptali, zda v nich existuje nezávislá množina či klika určité velikosti. Výstupem problému tedy byly vždy pouze hodnoty 1 (pravda) nebo 0 (nepravda). Takovým problémům se říká tzv. **rozhodovací problémy**.

Mimo tento typ se však v praxi často setkáme s **optimalizačními problémy**. Nejspíše nás nebude zajímat jen to, zda v grafu existuje klika určité velikosti, ale jak velká je největší klika. U problémů s číselným parametrem, které zde probíráme, lze optimalizační hodnotu získat opakovaným voláním rozhodovací varianty. Graf $G = (V, E)$ ponecháme stejný a měníme parametr k .

Uvažujme, že podprogram $\text{ExistsClique}(G, k)$ určuje pro vstupní graf G a číslo k , zda v něm existuje klika velikosti alespoň k . Pomocí něj pak umíme vyřešit i optimalizační verzi, viz 6.1. Podobně tak lze řešit

Algoritmus 6.1: Určení maximální kliky v grafu

Vstup: Graf $G = (V, E)$

```

1  $n \leftarrow |V|$ 
2 for  $k = n, n - 1, n - 2, \dots, 1, 0$  do
3   if  $\text{ExistsClique}(G, k) = 1$  then return  $k$ ;

```

Výstup: Velikost maximální kliky v grafu G

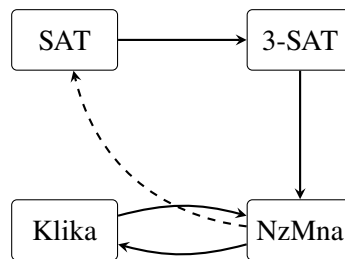
i ostatní problémy. Pro nás je však podstatné si uvědomit, že naše omezení na rozhodovací problémy má tedy hned dva důvody. Celkově nám to zjednodušuje pohled na danou problematiku a navíc od rozhodovacího problému k optimalizačnímu umíme přejít velice snadno.

Vraťme se tedy zpět k rozhodovacím variantám, které jsme již probírali. Zkusme si rozebrat rychlosti jednotlivých

převodů.

- $SAT \rightarrow 3-SAT$. Klauzuli s $k > 3$ literály rozložíme v čase $\mathcal{O}(k)$. Převod tedy potrvá maximálně lineárně dlouho v délce formule, tedy počtu jejích literálů n .
- $3-SAT \rightarrow SAT$. Formule v 3-CNF jsou již v CNF, takže použijeme totožný vstup. Jeho případné zkopírování zabere lineární čas.
- $3-SAT \rightarrow NzMna$. Sestrojený graf má vrcholy v počtu literálů formule, to potrvá $\mathcal{O}(n)$. Hran v grafu bude maximálně kvadraticky mnoho (tj. úplný graf), tedy $\mathcal{O}(n^2)$. Celkově $\mathcal{O}(n + n^2) = \mathcal{O}(n^2)$.
- $Klika \rightarrow NzMna$. Doplněk grafu má vrcholy původního grafu. Hran sestrojíme maximálně kvadraticky mnoho, tj. celkově $\mathcal{O}(n^2)$.
- $NzMna \rightarrow Klika$. Stejně jako u předchozího převodu.

Můžeme si všimnout, že všechny převody jsou polynomiální, tedy jak jsme uváděli v úvodu kapitoly, můžeme je považovat za „rychlé“, viz obrázek 11. Co však dodnes neumíme, je kterýkoliv z těchto problémů v polynomiálním čase řešit. Uvedené převody nám je ale dávají hezky do souvislosti. Pokud bychom uměli řešit kterýkoliv z nich rychle, pak bychom uměli rychle řešit všechny. Libovolný jiný bychom totiž převedli na ten, který rychle řešit umíme. Např. pokud bychom byli schopni řešit problém kliky v polynomiálním čase, mohli bychom řešit i 3-



Obrázek 11: Shrnutí převodů mezi jednotlivými problémy.

SAT: formuli polynomiálně převedeme na instanci nezávislé množiny, sestrojíme doplněk vzniklého grafu a na něm spustíme algoritmus pro kliku. Složení konečného počtu polynomiálních algoritmů je opět polynomiální algoritmus.

6.2 Ověřování problémů

Už jsme si zmínili, že žádné ze zmíněných problémů (ani mnohé jiné) neumíme řešit v polynomiálním čase, ač instance mnohých z nich dnes umíme řešit v uspokojivých časech. U hodně z nich však umíme rychle ověřit, zda je nějaké řešení správné. Uvažujme situaci, kdy máme zadanou logickou formuli φ a nějaké ohodnocení jejích proměnných. Zajímá nás, zda je dané ohodnocení splňující. To není ale vůbec těžké. Zkrátka danou formuli vyhodnotíme. To však již umíme provést rychle!

- Pokud je zadané ohodnocení splňující, pak je φ splnitelná.
- Pokud ohodnocení není splňující, pak to ještě nutně *neznamená*, že je *nesplnitelná*.

Všimněme si tedy, že nyní se naše otázka změnila.

- Původní otázka: *Je zadaná formule φ splnitelná?*
- Současná otázka: *Je zadané ohodnocení proměnných zadané formule φ splňující?*

Pojďme se podívat touto optikou ještě na jiný problém. U problému NzMna nás zajímalo, zda zadaný graf G obsahuje nezávislou množinu velikosti alespoň k . Nyní nás bude zajímat následující otázka: *Je zadaná množina N*

v grafu G nezávislá velikosti alespoň k ? Opět jsme přeformulovali otázku do podoby, kdy původně hledaný¹¹ objekt již máme zadáný a pouze ověřujeme, zda splňuje danou vlastnost. Zde je řešení opět velice jednoduché, zkrátka zkontrolujeme, zda je množina dostatečně velká a zda mezi žádnou dvojicí vrcholů nevede hrana. Viz algoritmus 6.2.

Algoritmus 6.2: Ověření nezávislé množiny

Vstup: Graf $G = (V, E)$, číslo $k \in \mathbb{N}$ a množina $N \subseteq V$
 // Množina N není dostatečně velká
 1 **if** $|N| < k$ **then return** 0;
 // Dvojice vrcholů je spojena hranou
 2 **foreach** $u \in N$ **do**
 3 **foreach** $v \in N; v \neq u$ **do**
 4 **if** $\{u, v\} \in E$ **then return** 0;
 // Množina N je nezávislá
 5 **return** 1

Časová složitost je zjevně polynomiální. Označíme $|N| = m$, přičemž jistě platí $m \leq |V|$. Vnější cyklus se provede m -krát a vnitřní cyklus $(m - 1)$ -krát (vrchol u jako jediný znovu nevybíráme). Při reprezentaci grafu maticí sousednosti ověříme existenci hrany v konstantním čase. Tedy celkově

$$\mathcal{O}(m(m - 1)) = \mathcal{O}(m^2 - m) = \mathcal{O}(m^2),$$

což je polynom. Tedy umíme rychle ověřit, zda je množina nezávislá.

Podobně si můžete rozmyslet, jak budou vypadat takové “ověřovací” algoritmy pro ostatní problémy. Co je ovšem podstatou tohoto výkladu? Obecně existují problémy, které jsme schopni řešit s určitou “náповědou”. Např. v případě nezávislé množiny si můžeme nechat napovědět nějakou podmnožinu vrcholů $N \subseteq V$ a nám jen stačí ověřit, zda je skutečně nezávislá a dostatečně velká. Podobně u problému SAT si necháme napovědět ohodnocení proměnných a nám pouze stačí ověřit, zda danou formuli splňuje.

Poznámka 6.1. U třídy NP požadujeme, aby náповěda měla délku nejvýše polynomiální ve velikosti původního vstupu. Úplný optimální postup pro n -diskové Hanojské věže má $2^n - 1$ tahů, takže už samotný jeho zápis je exponenciálně dlouhý. Takový zápis proto nemůže sloužit jako polynomiálně dlouhý certifikát.

6.3 Zavedení tříd složitosti

Předchozí úvahy nám dávají rozdělení problémů do dvou základních kategorií.

- Ty, které *umíme řešit v polynomiálním čase*
- a ty, u nichž *dokážeme v polynomiálním čase ověřit správnost řešení*.

Tím dostáváme tzv. *třídy složitosti* P a NP (také jim říkáme *třídy problémů*). Pojdme se na ně podívat.

P je třída rozhodovacích problémů řešitelných v polynomiálním čase.

Třída P tedy zachycuje všechny problémy, které jsou v jistém smyslu rychle řešitelné¹².

⁽¹¹⁾Formálně nelze přímo říci, že takový objekt vyloženě “hledáme”. Pouze nás zajímá, zda existuje.

⁽¹²⁾Asi by čtenáře mohlo napadnout, že např. algoritmus s časovou složitostí $\mathcal{O}(n^{1000})$ je jen stěží použitelný. Z druhé strany může být algoritmus s časovou složitostí $\mathcal{O}(1,001^n)$ prakticky použitelný pro určité velikosti dat, byť je exponenciální. Polynomiální čas je teoretická hranice s mnoha příjemnými vlastnostmi, nikoliv záruka praktické rychlosti každého konkrétního algoritmu.

Příklad 6.2. Příklady některých (rozhodovacích) problémů, které jsou řešitelné v polynomiálním čase.

- (i) *Existence cesty mezi dvěma vrcholy.* Lze řešit např. pomocí BFS nebo DFS¹³; oba algoritmy jsou polynomiální.
- (ii) *Existence podřetězce v textu.* Máme zadaný textový řetězec délky n a slovo délky k . Naivně můžeme otestovat všech $n - k + 1$ možných počátečních pozic a na každé porovnat nejvýše k znaků. Dostaneme polynomiální čas $\mathcal{O}((n - k + 1)k)$.
- (iii) *Existence prvku v poli.* Lze řešit např. sekvenčním vyhledáváním, které běží v čase $\mathcal{O}(n)$.

Všechny zmíněné případy problémů jsou řešitelné v čase $\mathcal{O}(n^k)$.

NP je třída rozhodovacích problémů, u nichž lze kladnou odpověď doložit polynomiálně dlouhým certifikátem a ten ověřit v polynomiálním čase.



Třídy P a NP zde zavádíme neformálně a přeskakujeme technické podrobnosti o kódování vstupů a výpočetním modelu. Pro naše potřeby si s tímto pohledem vystačíme.¹⁴



Písmeno „N“ ve zkratce NP neznamena „nepolynomiální“. Třída se jmenuje podle nedeterministického polynomiálního času; v našem výkladu jej zachycuje právě polynomiálně ověřitelný certifikát.

Problémy třídy NP jsou již trochu zajímavější, neboť již nutně nepožadujeme, aby byl problém rychle řešitelný. Podmínkou je pouze, že musíme být schopni rychle ověřit, zda zadaný objekt splňuje požadovanou vlastnost. Alternativně též můžeme na třídu NP nahlížet tak, že daný problém jsme schopni řešit rychle s určitou nápovědou.

Příklad 6.3. Příklady problémů ležících ve třídě NP jsme si již ukazovali v sekci 6.1, ale ještě jednou si je zde shrneme.

- (i) *SAT, 3-SAT.* Jako nápověda zde slouží ohodnocení proměnných. Stačí ověřit, zda je splňující vyhodnocením zadané formule.
- (ii) *Nezávislá množina.* Na vstupu dostaneme kromě grafu G a čísla k ještě nějakou množinu vrcholů N . Nám posléze stačí rozhodnout, zda N je nezávislá a platí $|N| \geq k$. Viz algoritmus 6.2.
- (iii) *Klika.* Opět dostaneme množinu vrcholů N . Postup podobný jako u NzMna.

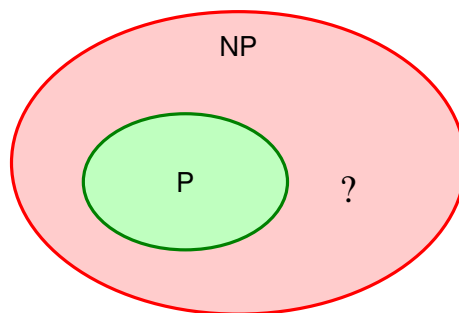
Může se na první pohled zdát, že třídy P a NP představují dva oddělené světy. Ve skutečnosti tomu tak není. Lze si totiž všimnout zajímavého vztahu, který má kupodivu jednoduché zdůvodnění.

Věta 6.4. Platí $P \subseteq NP$.

Věta 6.4 říká, že třída P leží uvnitř třídy NP, viz obrázek 12. Pokud totiž dokážeme nějaký problém $A \in P$ řešit v polynomiálním čase (tedy bez nápovědy), tím spíš jej dokážeme řešit rychle s nápovědou (např. algoritmus může nápovědu ignorovat a určit řešení přímo z původního vstupu). Tento fakt ovšem zahrnuje i potenciální variantu, že $P = NP$. To dodnes není známo a otázka, zda jsou třídy P a NP skutečně různé, zůstává nezodpovězena¹⁵. Obecně je však přijímána spíše hypotéza, že jsou tyto třídy různé.

⁽¹³⁾O DFS již víme, že nenalezne nutně nejkratší cestu, ale zde řešíme pouze její existenci.

⁽¹⁵⁾Problém P vs. NP patří mezi tzv. *Problémy tisíciletí*, které vyhlásil Clayův matematický institut v roce 2000. Na vyřešení každého z nich vypsál institut jeden milion amerických dolarů.



Obrázek 12: Schematická ilustrace vztahu $P \subseteq NP$; zda je inkluze ostrá, nevíme.

U všech zmíněných problémů v této kapitole s jistotou víme, že leží ve třídě NP , avšak nevíme, zda leží i ve třídě P , neboť pro žádný z nich neznáme polynomiální algoritmus. Avšak lze o nich ukázat, že jsou tyto problémy (a mnohé jiné) v jistém smyslu “ty nejtěžší” ve třídě NP .

Definice 6.5 (NP-úplný problém). Problém A nazveme NP-úplným, pokud $A \in NP$ a zároveň je na něj polynomiálně převoditelný každý problém z NP . Tzn.

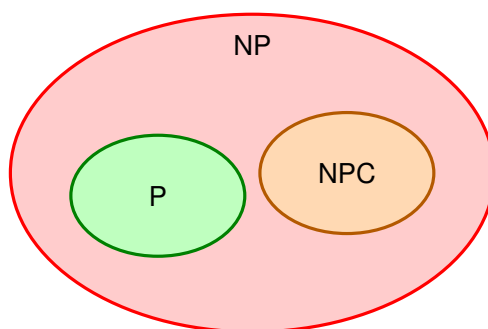
$$\forall L \in NP : L \rightarrow A.$$

Druhá podmínka v definici 6.5 se možná zdá hodně silná. Existuje vůbec takový problém? Ač se to může zdát nepravděpodobné, tak v roce 1971 dokázal americký informatik Stephen Cook překvapivou větu.

Věta 6.6. Problém SAT je NP-úplný.

Důkaz toho tvrzení si zde ukazovat nebudeme, neboť je zcela nad rámec tohoto výkladu. Tento výsledek má v této teorii velice silný důsledek. Pokud by se ukázalo, že nějaký NP-úplný problém A leží ve třídě P , tak by z toho již plynulo, že $P = NP$. Každý problém z NP bychom mohli jednoduše polynomiálně převést na A a ten pak vyřešit opět polynomiálně. Toto je přesně důvod, proč problém SAT hraje v současné informatice takovou důležitou roli, neboť se historicky ukázalo, že pomocí něj lze řešit kterýkoliv jiný problém¹⁶ v NP .

NP-úplných problémů existuje více. Třída takových problémů se označuje jako NPC (z anglického *NP-Complete*) a platí $NPC \subseteq NP$, viz obrázek 13. Pokud je NP-úplný problém A polynomiálně převoditelný na problém B , pak je B NP-těžký; aby byl také NP-úplný, musíme navíc ověřit $B \in NP$. Převody z této kapitoly společně se snadným ověřením certifikátů ukazují, že *SAT*, *3-SAT*, *NzMna* a *Klika* jsou NP-úplné.



Obrázek 13: Obvyklé schematické znázornění vztahu P , NP a NPC za předpokladu $P \neq NP$.

Jak už jsme si ale uvedli, to zda platí $P = NP$, stále nevíme. Jaké by to ale mělo důsledky v praxi? Pojďme se alespoň na chvíli zamyslet nad oběma variantami.

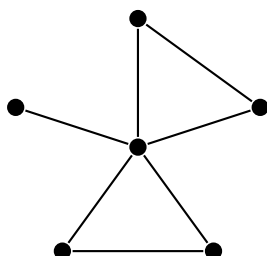
⁽¹⁶⁾Proto se někdy též říká, že NP-úplné problémy tvoří v jistém smyslu ty “nejtěžší” v NP , neboť pokud umíme řešit nějaký z nich, pak umíme vyřešit všechny.

- $P = NP$. Každý rozhodovací problém z NP by měl polynomiální algoritmus a u mnoha běžných NP-úplných úloh bychom pomocí opakovaných dotazů dokázali polynomiálně nalézt i řešení. Samotná rovnost by však nezaručovala prakticky malý exponent ani automaticky nezrušila úplně všechny druhy šifrování. Zásadně by ovšem zpochybnila běžné kryptografické předpoklady založené na výpočetní obtížnosti.
- $P \neq NP$. Žádný NP-úplný problém by neležel v P, takže by pro SAT, NzMna nebo Kliku neexistoval polynomiální algoritmus. Ani tato nerovnost sama o sobě však nestačí k existenci bezpečné kryptografie; ta vyžaduje silnější předpoklady o obtížnosti konkrétních problémů.

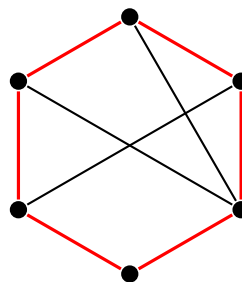
6.4 Další NP-úplné problémy

K závěru této kapitoly si ještě uvedeme nějaké další NP-úplné problémy a k nim i nějaké příklady jejich aplikací (resp. výskytů). NP-úplnost si zde však již dokazovat pro žádný z nich nebudeme. Pro žádný z těchto problémů není známý polynomiální algoritmus.

- (i) *Hamiltonovská kružnice*. Máme zadaný graf $G = (V, E)$. Zajímá nás, zda v něm existuje kružnice, která projde každý vrchol právě jednou. Viz obrázky 14a a 14b. Příbuzný problém *obchodního cestujícího* navíc



(a) Příklad grafu, kde neexistuje hamiltonovská kružnice



(b) Graf a jeho hamiltonovská kružnice

pracuje s cenami hran a hledá co nejlevnější okružní trasu. Právě tato optimalizační varianta se objevuje v logistice.

- (ii) *Součet podmnožiny*. Máme zadaný seznam přirozených čísel M a číslo $S \in \mathbb{N}$. Existuje výběr prvků seznamu, jehož součet je roven S ? Např. pro množinu $M = \{1, 3, 10, 11, 20\}$ (ne)existují součty uvedené v tabulce 1. Rychlé řešení součtu podmnožiny by pomohlo např. s optimalizací plánování nákladů rozpočtem

Součet S	13	2	22	16
Existence	✓	✗	✓	✗

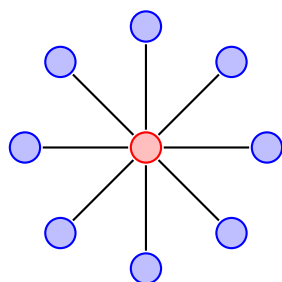
Tabulka 1: Tabulka součtů množiny M

(např. ve firmě). Dále se též používá u tzv. *jednosměrných funkcí* v kryptografii.

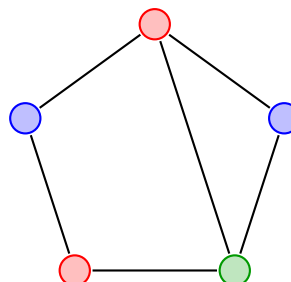
- (iii) *Barvení grafu*. Chceme zjistit, zda pro zadaný graf je možné jeho vrcholy obarvit k barvami (formálně přidělíme každému vrcholu číslo od 1 do k) tak, aby vrcholy stejné barvy nebyly nikdy spojeny hranou? Pro příklady viz obrázky 15a a 15b. Pro $k = 2$ lze barvení rozhodnout polynomiálně, ale pro každé pevné $k \geq 3$ je problém NP-úplný.
- (iv) *Problém batohu*. Jsou dány předměty s váhami a cenami a kapacita batohu. Chceme najít co nejdražší podmnožinu předmětů tak, abychom nepřekročili kapacitu batohu. V rozhodovací variantě se ptáme, zda existuje podmnožina předmětů s celkovou cenou alespoň C a celkovou vahou nejvýše W .

Formálněji máme váhy předmětů $w_1, \dots, w_n$ a jejich ceny $c_1, \dots, c_n$, přičemž hledáme hodnoty $x_1, \dots, x_n \in \{0, 1\}$ tak, aby platilo

$$\sum_{i=1}^n c_i x_i \geq C \quad \text{a} \quad \sum_{i=1}^n w_i x_i \leq W.$$



(a) Barvení grafu pro $k = 2$.



(b) Barvení grafu pro $k = 3$.

Problém batohu se též objevuje v kryptografii.

- (v) *Dva loupežníci*. Ptáme se, zda lze zadaný seznam kladných celých čísel rozdělit na dvě části se stejným součtem.¹⁷ Například

$$L = \{1, 3, 4, 6, 8, 10\}$$

lze rozdělit na $L_1 = \{6, 10\}$ a $L_2 = \{1, 3, 4, 8\}$; obě části mají součet 16. S příbuznými úlohami se setkáme při vyvažování zátěže mezi procesory.

⁽¹⁷⁾Loupežníci tedy řeší, jak spravedlivě rozdělit lup.